

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi

DATABASE MANAGEMENT SYSTEM

RELATIONAL DATABASE VS. NoSQL DATABASE

BACHELORS OF COMPUTER APPLICATION- III

NAME: PREKSHA V

REGISTER NUMBER: 2511021240063

SUBMITTING TO,

RENITA JOY R

DBMS ASSIGNMENT - 4

1. Introduction

Databases store, organize, and retrieve information used by applications and organizations. The two broad database approaches discussed in this report are relational databases and NoSQL databases. Relational databases organize data into tables with defined relationships, while NoSQL databases use models such as document, key-value, wide-column, or graph structures.

Neither approach is universally better. The correct choice depends on the type of data, transaction requirements,workload,scale, development needs, and operational constraints.

2. Relational Databases

Arelational databasestores datain tables consisting of rows and columns. Tables are connected through primary keys and foreign keys, and SQL is commonly used to define, query, and update the data.

Relational systems typically support transactions with ACID properties: atomicity, consistency, isolation, and durability.

Examples: MySQL, PostgreSQL, Microsoft SQL Server, and Oracle Database.

Typical uses: banking transactions, university records, inventory, payroll, booking systems, and other applications whererelationshipsand reliable multi-step transactions matter.

3. NoSQL Databases

NoSQL refers toafamilyofnon-relational database technologies. Depending on the product, data may be stored as documents, key-value pairs, wide-column records, or graph structures. Many NoSQL systems are designed for flexible schemas, distributed storage, and workloads that need to scale across multiple machines.

Examples: MongoDB (document), Redis (key-value), Apache Cassandra (wide-column), and Neo4j (graph). NoSQL systems differ considerably; their consistency and transaction capabilities depend on the product and configuration.

Typical uses: content catalogs, application sessions, event streams, high-volume telemetry, recommendation relationships, and applications whose data shape changes frequently.

4. Comparative Analysis

Factor	Relational database	NoSQL database
Data model	Structured tables with predefined columns and relationships.	Document, key-value, wide-column, or graph models; structure varies by system.
Scalability	Often scales vertically by adding resources to one server; many systems also support read replicas, partitioning, and distributed deployments.	Often designed for horizontal scaling across multiple nodes, though implementation and ease of scaling vary by product.
Consistency	Commonly provides strong transactional consistency and ACID transactions. Exact isolation and distributed behavior depend on the system.	Consistency models vary. Some systems offer tunable or eventual consistency; others support strong consistency and transactions.
Performance	Very effective for joins, complex queries, reporting, and transactional workloads with related data.	Can be very fast for access patterns it is designed for, such as key lookups, document retrieval, or large distributed writes.
Schema flexibility	Usually uses a defined schema; schema changes may require migration and validation.	Often supports flexible or evolving data structures, especially document databases.
Relationships	Foreign keys and joins are core features in many relational systems.	Relationships may be embedded, referenced, or modeled as graph edges; joins may be limited or handled differently.
Transactions	Mature support for multi-row and multi-table transactions is common.	Transaction support varies by product; some support multi-document or distributed transactions, sometimes with trade-offs.
Best fit	Structured data, complex relationships, financial records, and strict transaction requirements.	Large distributed workloads, flexible data, high write volume, or access patterns suited to a particular NoSQL model.

Important note: These are general tendencies, not absolute rules. Modern relational databases can scale horizontally in some deployments, and several NoSQL databases provide strong consistency and transactions. Thespecific product, configuration, and workload must be evaluated.

5. Scalability, Consistency, and Performance in Practice

Scalability: Vertical scaling means increasing theresources of oneserver. Horizontalscalingmeans distributing data or workload across multiple servers. NoSQL systems are often selected when horizontal distribution is central to the design, while relational databases remain suitable for many high-scale applications with replication, partitioning, and careful architecture.

Consistency: Strong consistency means users see updates according to the database's defined consistency guarantees. Eventual consistency allows replicas to temporarily differ but converge when updates stop. Organizations must decide whether a temporary difference is acceptable. A bank balance generally needs strict transaction correctness; a social-media like count may tolerate a short delay. Performance: Performance depends on query patterns, indexes, data volume, network latency, hardware, and design. Relational joins are useful when many related entities must be queried together. A document database may be more efficient when an application usually retrieves a whole self-contained document. Benchmarking with realistic data is more reliable than choosing based on a database label alone.

6. Real-World Examples

Thefollowing examples illustrate suitable use cases. They describe common technology fit and publicly known product use, not a claim that every part of each organization uses only one database.

Example	Possible / known database fit	Why it fits
Banking and payments	Relational systems such as Oracle Database, PostgreSQL, or SQL Server are common choices for core transaction records.	Balances, transfers, and account records require integrity, constraints, and reliable multi-step transactions.
E-commerce orders	A relational database can manage customers, orders, order items, inventory, and payments.	These entities have clear relationships, and order processing benefits from transactions and referential integrity.
MongoDB document applications	MongoDB is a real-world document database used for applications with JSON-like records and evolving data structures.	Documents can represent related fields together, which may simplify development for document-oriented access patterns.
Redis caching and sessions	Redis is commonly used for cache data, short-lived sessions, and fast key-based access.	Key-value operations can provide low-latency reads and writes for suitable workloads; data persistence and eviction settings must be planned.
Apache Cassandra distributed data	Cassandra is designed for distributed data workloads across clusters and is used in high-volume, highly available applications.	Its data model and partitioning can support large write-heavy workloads when queries are designed around access patterns.
Neo4j graph applications	Neo4j is used for graph-oriented data such as recommendation links, fraud relationships, and network analysis.	Nodes and relationships are first-class concepts, making connected-path queries natural.

7. Why Organizations Choose Different Database Technologies

Organizationsselect adatabasebasedontheproblemtheyneedtosolveratherthansimply following a trend. Key decision factors include:

1. Data structure: Highly structured records with many relationships often fit relational tables. Flexible or nested records may fit a document database; connected networks may fit a graph database.

2. Transaction and correctness needs: Financial transactions, inventory updates, and booking confirmations often need strong integrity and transactional guarantees.

3. Scale and availability: Systems serving users across regions or processing large volumes of events may prioritize distributed architecture and availability.

4. Query patterns: Complex joins and ad hoc reporting often favor relational systems. Predictable key lookups, document retrieval, or graph traversal may favor a specialized NoSQL model.

5. Flexibility and speed of development: A flexible schema can help teams evolve application data quickly, but validation rules still need to be designed.

6. Cost and skills: Licensing, hosting, maintenance, monitoring, backup, migration, and staff experience all affect the total cost of ownership.

7. Existing systems and compliance: Organizations may need to integrate with current tools, meet audit

8. Applying the Comparison to a Cab Booking System

For the Cab Booking Management System in theearlier assignments, a relational databasesuch as MySQL or PostgreSQL is a suitable primary choice. CUSTOMER, DRIVER, CAB, BOOKING, TRIP, PAYMENT, and RATING have clear relationships, and foreign keys can protect those links. Booking confirmation and payment records also benefit from transaction controls.

A NoSQL database could complement the relational database for specific needs—for example, storing high-volume driver location updates, caching frequently accessed driver availability, or handling flexible event logs. This hybrid approach is optional and adds operational complexity, so it should be used only when the workload justifies it.

9. Advantages and Limitations

Approach	Advantages	Limitations
Relational	Strong data integrity; SQL and joins; mature transaction support; well suited to structured records and reporting.	Schema migrations may require planning; joins and distributed scaling can require careful tuning at very large scale.
NoSQL	Flexible data models; can suit horizontal distribution and high-volume access patterns; specialized models fit particular workloads.	Products differ widely; relationships and cross-record consistency may require more application design; data duplication can complicate updates.

10. Decision Guide

Requirement	Likely starting point	Reason
Multiple related tables and strict transactions	Relational	Keys, constraints, joins, and transactions help maintain integrity.
Flexible nested product or content records	Document NoSQL	A document model may align with the way the application reads and writes data.
Fast cache or session lookup by key	Key-value NoSQL	Simple key-based operations suit caching and session workloads.
Large distributed write workload with predictable queries	Wide-column NoSQL	Partitioned data models can be optimized for high-volume distributed access.
Deeply connected relationships and path queries	Graph NoSQL	Relationships are stored and queried as graph connections.